



NVDEC Video Decoder API Programming Guide

Release 13.1

NVIDIA Corporation

May 22, 2026

Contents

1 Overview	1
1.1 Supported Codecs	1
2 Video Decoder Capabilities	3
3 Video Decoder Pipeline	5
4 Using NVIDIA Video Decoder (NVDEC API)	7
4.1 Video Parser	7
4.1.1 Creating a parser	7
4.1.2 Parsing the packets	8
4.1.3 Destroying parser	9
4.2 Video Decoder	9
4.2.1 Querying decode capabilities	9
4.2.2 Creating a Decoder	11
4.2.3 Decoding the frame/field	13
4.2.4 Preparing the decoded frame for further processing	13
4.2.5 Getting histogram data buffer	15
4.2.6 Querying the decoding status	16
4.2.7 Reconfiguring the decoder	16
4.2.8 Destroying the decoder	17
4.3 Run-time dynamic linking of NVIDIA libraries	17
4.3.1 Run-time dynamic linking	17
4.3.2 Getting function pointers	18
4.4 Writing an Efficient Decode Application	20
4.5 Allocating decode surfaces dynamically	21
4.6 Getting decode statistics from the decoder	22
4.6.1 Enabling decode statistics	22
4.6.2 Retrieving decode statistics	23
4.6.3 Decode statistics structure	24
4.6.4 Sample application flow	25
4.7 Random Frame Access and Seeking	25
4.7.1 Understanding the Seek Architecture	26
4.7.2 Using NvVideoDecoder for Frame Access	26
4.7.3 Enabling Open GOP Support	28
4.7.4 PTS-Based Frame Skipping Optimization	29
4.7.5 Handling Non-Seekable Streams	29
4.7.6 Decoder Reconfiguration for Multiple Videos	30
4.7.7 Using AppDecVideoDecoder Sample Application	31
5 NVDEC Direct Output to Block-Linear CUDA Arrays	33
5.1 Key Advantages	33
5.2 Supported Opaque Surface Formats	33
5.3 Step-by-Step Usage	34

5.4	Reference: Traditional vs Opaque Data Flow	34
-----	--	----

Chapter 1. Overview

NVIDIA GPUs - beginning with the NVIDIA® Fermi™ generation and NVIDIA Jetson platforms contain a video decoder engine (referred to as NVDEC in this document) which provides fully-accelerated hardware video decoding capability. NVDEC can be used for decoding bitstreams of various formats: AV1, H.264, HEVC (H.265), VP8, VP9, MPEG-1, MPEG-2, MPEG-4 and VC-1. NVDEC runs completely independently of compute/graphics engine.

NVIDIA provides software API and libraries for programming NVDEC. The software API, hereafter referred to as NVDEC API, lets developers access the video decoding features of NVDEC and interoperate NVDEC with other engines on the GPU.

NVDEC decodes the compressed video streams and copies the resulting YUV frames to video memory (in GPUs) or system memory (in Jetson platforms). With frames in video/system memory, video post processing can be done using CUDA. The NVDEC API also provides CUDA-optimized implementation of commonly used post-processing operations such as scaling, cropping, aspect ratio conversion, de-interlacing and color space conversion to many popular output video formats. The client can choose to use the CUDA-optimized implementations provided by the NVDEC API for these post-processing steps or choose to implement their own post-processing on the decoded output frames.

Decoded video frames can be presented to the display with graphics interoperability for video playback, passed directly to a dedicated hardware encoder (NVENC) for high-performance video transcoding, used for GPU accelerated inferencing or consumed further by CUDA or CPU-based processing.

1.1. Supported Codecs

The codecs supported by NVDEC API are:

- ▶ MPEG-1,
- ▶ MPEG-2,
- ▶ MPEG-4,
- ▶ VC-1 (Simple/Main/Advanced Profile),
- ▶ H.264 (AVCHD) Baseline/Main/High/High10 (exclude MBAFF)/High422 (exclude MBAFF) Profile,
- ▶ H.265 (HEVC) (Main/Main10/Main12 Profile, Main 4:2:2/4:4:4 10/12 profile (exclude YUV400)),
- ▶ VP8,
- ▶ VP9 (8-bit, 10-bit, and 12-bit),
- ▶ AV1 Main profile,
- ▶ Hybrid (CUDA + CPU) JPEG

Note

The Hybrid (CUDA + CPU) JPEG codec is not supported on Jetson platforms.

Refer to Chapter 2 for complete details about the video capabilities for various GPUs and Jetson platforms.

Chapter 2. Video Decoder Capabilities

This chapter provides detailed information about the video decoder capabilities for each GPU architecture and Jetson platform. *NVDEC Capabilities* table specifies the maximum and minimum supported resolutions, supported profiles, and codec levels for H.264, HEVC, VP9, and AV1 to verify hardware support for required video formats and resolutions.

MPEG-1 & MPEG-2

- ▶ Max Resolution and Min Resolution supported on all GPUs is 4080x4080 and 48x16 respectively

VC-1 & MPEG-4

- ▶ Max Resolution and Min Resolution supported on all GPUs is 2048x1024 & 1024x2048 and 48x16 respectively

VP8

- ▶ Max Resolution and Min Resolution supported on all GPUs is 4096x4096 and 48x16 respectively

Table 1: NVDEC Capabilities

GPU Arch / Jetson Platform	H.264/AVCHD	H.265/HEVC	VP9	AV1
Turing/GA100	Max: 4096x4096	Max: 8192x8192	Max: 8192x8192 ¹	Unsupported
	Min: 48x16	Min: 144x144	Min: 128x128	
	Profile: Baseline, Main, High profile up to Level 5.1	Profile: Main profile up to Level 6.0, main10 and main12 profile	Profile: Profile 0, 10-bit and 12-bit decoding	
Hopper	Max: 4096x4096	Max: 8192x8192	Max: 8192x8192	Unsupported
	Min: 48x16	Min: 144x144	Min: 128x128	

continues on next page

Table 1 – continued from previous page

GPU Arch / Jetson Platform	H.264/AVCHD	H.265/HEVC	VP9	AV1
	Profile: Baseline, Main, High profile up to Level 5.1	Profile: Main, main10 and main12 profile up to Level 6.0, 444 chroma format support	Profile: Profile 0, 10-bit and 12-bit decoding	
GA10x/AD10x	Max: 4096x4096 Min: 48x16	Max: 8192x8192 Min: 144x144	Max: 8192x8192 Min: 128x128	Max: 8192x8192 Min: 128x128
	Profile: Baseline, Main, High profile up to Level 5.1	Profile: Main, main10 and main12 profile up to Level 6.0, 444 chroma format support	Profile: Profile 0, 10-bit and 12-bit decoding	Profile: Profile 0 up to level 6.0
Blackwell / Jetson Thor	Max: 8192x8192 Min: 48x64	Max: 8192x8192 Min: 144x144	Max: 8192x8192 Min: 128x128	Max: 8192x8192 Min: 128x128
	Profile: Baseline, Main, High up to Level 6.2 High 10 and High 422 (exclude MBAFF) up to Level 6.2	Profile: Main, main10, main12, Main 4:2:2/4:4:4 10/12 profile (exclude YUV400) Level up to 6.0	Profile: Profile 0, 10-bit and 12-bit decoding	Profile: Profile 0 up to level 6.0

- ¹: VP9 10-bit and 12-bit decoding is supported on all Turing GPUs and GA100
- **Max** in the table above refers to the maximum resolution supported by the hardware video decoder for the given GPU architecture or Jetson platform.
- **Min** in the table above refers to the minimum resolution supported by the hardware video decoder for the given GPU architecture or Jetson platform.

Chapter 3. Video Decoder Pipeline

This chapter describes the video decoder pipeline architecture and the workflow for decoding video using the NVDECOD API. It explains the decoder components and the sequence of API calls required for video decoding.

Decoder pipeline consists of three major components - Demuxer, Video Parser, and Video Decoder. The components are not dependent on each other and hence can be used independently. NVDECOD API provides APIs for NVIDIA video parser and NVIDIA video decoder. Of these, NVIDIA video parser is purely a software component and users can implement their own parser in place of NVIDIA video parser, if required.

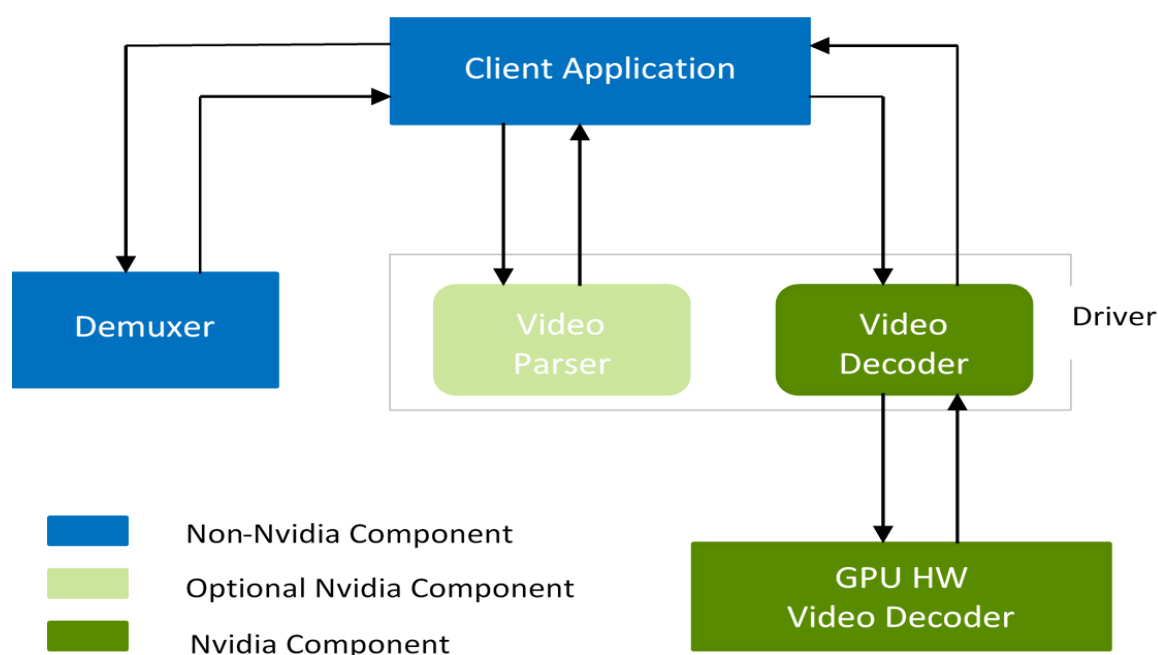


Fig. 1: Figure 1. Video decoder pipeline using NVDECOD API

At a high level the following steps should be followed for decoding any video content using NVDECOD API:

1. Create a CUDA context.
2. Query the decode capabilities of the hardware decoder.
3. Create the decoder instance(s).

4. De-Mux the content (like .mp4). This can be done using third party software like FFMPEG.
5. Parse the video bitstream using parser provided by NVDEC API or third-party parser such as FFmpeg.
6. Kick off the Decoding using NVDEC API.
7. Obtain the decoded YUV for further processing.
8. Query the status of the decoded frame.
9. Depending on the decoding status, use the decoded output for further processing like rendering, inferencing, postprocessing etc.
10. If the application needs to display the output,
 - ▶ Convert decoded YUV surface to RGBA.
 - ▶ Map RGBA surface to DirectX or OpenGL texture.
 - ▶ Draw texture to screen.
11. Destroy the decoder instance(s) after the completion of decoding process.
12. Destroy the CUDA context.

The above steps are explained in the rest of the document and demonstrated in the sample application(s) included in the Video Codec SDK package.

Chapter 4. Using NVIDIA Video Decoder (NVDECODE API)

All NVDECODE APIs are exposed in two header-files: `cuvidec.h` and `nvcuvid.h`. These headers can be found under `Interface` folder in the Video Codec SDK package. The samples in NVIDIA Video Codec SDK statically load the library (which ships as a part of the SDK package for windows) functions and include `cuvidec.h` and `nvcuvid.h` in the source files. The Windows DLL `nvcuvid.dll` is included in the NVIDIA display driver for Windows. The Linux library `libnvcuvid.so` is included with NVIDIA display driver for Linux. For Jetson platforms, the library `libnvcuvid.so` is included in the Jetson Linux.

The following sections in this chapter explain the flow that should be followed to accelerate decoding using NVDECODE API.

4.1. Video Parser

4.1.1. Creating a parser

Parser object can be created by calling `cuvidecCreateVideoParser()` after filling the structure `CUVIDPARSERPARAMS`. The structure should be filled up with following information about the stream to be decoded:

- ▶ `CodecType`: must be from enum `cudaVideoCodec`, indicating codec type of content like H.264, HEVC, VP9 etc.
- ▶ `ulMaxNumDecodeSurfaces`: This is number of surfaces in parser's DPB (decode picture buffer). This value may not be known at the parser initialization time and can be set to a dummy number like 1 to create parser object. Application must register a callback `pfnSequenceCallback` with the driver, which is called by the parser when the parser encounters the first sequence header or any changes in the sequence. This callback reports the minimum number of surfaces needed by parser's DPB for correct decoding in `CUVIDEOFORMAT::min_num_decode_surfaces`. The sequence callback may return this value to the parser if wants to update `CUVIDPARSERPARAMS::ulMaxNumDecodeSurfaces`. The parser then overwrites `CUVIDPARSERPARAMS::ulMaxNumDecodeSurfaces` with the value returned by the sequence callback, if return value of the sequence callback is greater than 1 (see description about `pfnSequenceCallback` below). Therefore, for optimum memory allocation, decoder object creation should be deferred until `CUVIDPARSERPARAMS::ulMaxNumDecodeSurfaces` is known, so that the decoder object can be created with required number of buffers, such that `CUVIDDECODECREATEINFO::ulNumDecodeSurfaces = CUVIDPARSERPARAMS::ulMaxNumDecodeSurfaces`.
- ▶ `ulClockRate`: is timestamp units in Hz (0=default=100000000Hz)

- ▶ **ulErrorThreshold**: controls non-compliance bitstream checks in parser. Its valid range is 0 to 100. 0 means strict check and parser will return error if found any non-compliance or error and 100 means ignore all non-compliance bitstream checks in parser.
- ▶ **ulMaxDisplayDelay**: Max display callback delay. 0 = no delay
- ▶ **bAnnexb**: must be set to 1 for AV1 annexB streams
- ▶ **pfnSequenceCallback**: Application must register a function to handle any sequence change. Parser triggers this callback for initial sequence header or when it encounters a video format change. Return value from sequence callback is interpreted by the driver as follows:
 - ▶ 0: fail
 - ▶ 1: succeeded, but driver should not override **CUVIDPARSER-
PARAMS::ulMaxNumDecodeSurfaces**
 - ▶ >1: succeeded, and driver should override **CUVIDPARSER-
PARAMS::ulMaxNumDecodeSurfaces** with this return value
- ▶ **pfnDecodePicture**: Parser triggers this callback when bitstream data for one frame is ready. In case of field pictures, there may be two decode calls per one display call since two fields make up one frame. Return value from this callback is interpreted as:
 - ▶ 0: fail
 - ▶ ≥ 1 : succeeded
- ▶ **pfnDisplayPicture**: Parser triggers this callback when a frame in display order is ready. Return value from this callback is interpreted as:
 - ▶ 0: fail
 - ▶ ≥ 1 : succeeded
- ▶ **pfnGetOperatingPoint**: Parser triggers this callback to get operating point of an AV1 scalable stream. Parser picks default operating point as 0 and outputAllLayers flag as 0 if **pfnGetOperatingPoint** is not set or return value is -1 or invalid operating point. Return value from this callback is interpreted as:
 - ▶ < 0: fail
 - ▶ ≥ 0 : succeeded (bit 0-9: currOperatingPoint, bit 10-10: bOutputAllLayer)
- ▶ **pfnGetSEIMsg**: Parser triggers this callback in decode order when all the unregistered user SEI messages or Metadata OBUs are parsed for a frame. Currently this callback is supported for H264, HEVC and AV1 codecs. Return value from this callback is interpreted as:
 - ▶ 0: fail
 - ▶ ≥ 1 : succeeded

4.1.2. Parsing the packets

Bitstream extracted from demultiplexer along with its length and some other auxiliary info like timestamp, flags is packed into struct **CUVIDSOURCEDATAPACKET**, called as packet. This packet is fed into parser using **cuvidParseVideoData()**. This packet is initialized as:

- ▶ **flags**: These flags are set by application and interpreted by parser as below:
 - ▶ **CUVID_PKT_ENDOFSTREAM**: MUST be set with last packet for this stream. Parser will trigger display callback for all pending buffers in the display queue.

- ▶ **CUVID_PKT_TIMESTAMP**: indicate that timestamp in packet is valid.
- ▶ **CUVID_PKT_DISCONTINUITY**: should be set if there is any discontinuity like packet after seek.
- ▶ **CUVID_PKT_ENDOFPICTURE**: MUST be set when packet contains exactly one frame or one field data. NALU based codecs have one frame latency for decode callback as parser detects frame boundary when some non-VCL NALU are received (that belong to next frame). This flag will force parser to skip this boundary check and trigger decode callback immediately. If packet has incomplete data, decode callback will get triggered with partial frame data. If packet has more than one frame data, parser will trigger decode callback for first frame data. Rest of the NALU will get dropped.
- ▶ **CUVID_PKT_NOTIFY_EOS**: If this flag is set along with **CUVID_PKT_ENDOFSTREAM**, an additional (dummy) display callback will be invoked with null value of **CUVIDPARSERDISPINFO** which should be interpreted as end of the stream.
- ▶ **payload_size**: represents number of bytes in payload
- ▶ **payload**: points to bitstream memory buffer
- ▶ **timestamp**: Presentation time stamp (10MHz clock), only valid if **CUVID_PKT_TIMESTAMP** flag is set

Parser triggers callbacks registered while creating parser object synchronously from within `cuvidParseVideoData()`, whenever a corresponding condition is hit like `pfnSequenceCallback` when there is change in sequence parameters or `pfnDecodePicture` when frame is ready to be decoded. If the callback returns failure, it will be propagated by `cuvidParseVideoData()` to the application.

The decoded result gets associated with a picture-index value in the **CUVIDPICPARAMS** structure, which is also provided by the parser. This picture index is later used to map the decoded frames to CUDA memory.

4.1.3. Destroying parser

The user needs to call `cuvidDestroyVideoParser()` to destroy the parser object and free up all the allocated resources.

4.2. Video Decoder

4.2.1. Querying decode capabilities

The API `cuvidGetDecoderCaps()` lets users query the capabilities of underlying hardware video decoder.

As illustrated in *NVDEC Capabilities*, different GPUs have hardware decoders with different capabilities. Therefore, to ensure your application works on all generations of GPU hardware, it is highly recommended that the application queries the hardware capabilities and makes appropriate decision based on presence/absence of the desired capability/functionality.

The API `cuvidGetDecoderCaps()` lets users query the capabilities of underlying hardware video decoder. Calling thread should have a valid CUDA context associated.

The client needs to fill in the following fields of **CUVIDDECDECAPS** before calling `cuvidGetDecoderCaps()`.

- ▶ eCodecType: Codec type (AV1, H.264, HEVC, VP9, JPEG etc.)
- ▶ eChromaFormat: enum cudaVideoChromaFormat (Monochrome, 420, 422, 444)
- ▶ nBitDepthMinus8: 0 for 8-bit, 2 for 10-bit, 4 for 12-bit

When `cuidGetDecoderCaps()` is called, the underlying driver fills up the remaining fields of `CUVID_DECODECAPS`, indicating the support for the queried capabilities, supported output formats and the maximum and minimum resolutions the hardware supports.

The following pseudo-code illustrates how to query the capabilities of NVDEC.

```
CUVIDDECODECAPS decodeCaps = {};
// set IN params for decodeCaps
decodeCaps.eCodecType = cudaVideoCodec_HEVC; //HEVC
decodeCaps.eChromaFormat = cudaVideoChromaFormat_420; //YUV 4:2:0
decodeCaps.nBitDepthMinus8 = 2; // 10 bit
result = cuvidGetDecoderCaps(&decodeCaps);
```

Returned parameters from API can be interpreted as below to validate if content can be decoded on underlying hardware:

```
// Check if content is supported
if (!decodeCaps.bIsSupported){
    NVDEC_THROW_ERROR("Codec not supported on this GPU", CUDA_ERROR_NOT_
↳SUPPORTED);
}
// validate the content resolution supported on underlying hardware
if ((coded_width > decodeCaps.nMaxWidth) ||
    (coded_height > decodeCaps.nMaxHeight)){
    NVDEC_THROW_ERROR("Resolution not supported on this GPU", CUDA_ERROR_
↳NOT_SUPPORTED);
}
// Max supported macroblock count CodedWidth*CodedHeight/256 must be <=
↳nMaxMBCount
if ((coded_width>>4)*(coded_height>>4) > decodeCaps.nMaxMBCount){
    NVDEC_THROW_ERROR("MBCount not supported on this GPU",
    CUDA_ERROR_NOT_SUPPORTED);
}
```

In most situations, bit-depth and chroma subsampling to be used at the decoder output is same as that at the decoder input (i.e. in the content). In certain cases, however, it may be necessary to have the decoder produce output with bit-depth and chroma subsampling different from that used in the input bitstream. In general, it's always a good idea to first check if the desired output bit-depth and chroma subsampling format is supported before creating the decoder. This can be done in the following way:

```
// Check supported output format
if (decodeCaps.nOutputFormatMask & (1<<cudaVideoSurfaceFormat_NV12)){
    // Decoder supports output surface format NV12
}
if (decodeCaps.nOutputFormatMask & (1<<cudaVideoSurfaceFormat_P016)){
    // Decoder supports output surface format P010
}
.....
```

The API `cuidGetDecoderCaps()` also returns histogram related capabilities of underlying GPU. Histogram data is collected by NVDEC during the decoding process resulting in zero performance penalty.

NVDEC computes the histogram data for only the luma component of decoded output, not on post-processed frame(i.e. when scaling, cropping, etc. applied). In case of AV1 when film grain is enabled, histogram data is collected on the decoded frame prior to the application of the film grain.

```
// Check if histogram is supported
if (decodecaps.bIsHistogramSupported){
    nCounterBitDepth = decodecaps.nCounterBitDepth; // histogram counter
    ↪bit depth
    nMaxHistogramBins = decodecaps.nMaxHistogramBins; // Max number of
    ↪histogram bins
}
.....
```

Histogram data is calculated as : `Histogram_Bin[pixel_value >> (pixel_bitDepth - log2(nMaxHistogramBins)) ]++;`

4.2.2. Creating a Decoder

Before creating the decoder instance, user needs to have a valid CUDA context which will be used in the entire decoding process.

The decoder instance can be created by calling `cuidCreateDecoder()` after filling the structure `CUVIDDECODERCREATEINFO`. The structure `CUVIDDECODERCREATEINFO` should be filled up with the following information about the stream to be decoded:

- ▶ **CodecType**: must be from enum `cudaVideoCodec`. It represents codec type of content like H.264, HEVC, VP9 etc.
- ▶ **ulWidth, ulHeight**: coded width and coded height in pixels.
- ▶ **ulMaxWidth, ulMaxHeight**: max width and max height that decoder support in case of resolution change. When there is resolution change (new resolution $\leq$ ulMaxWidth, ulMaxHeight) in video stream, app can reconfigure decoder using `cuidReconfigureDecoder()` API instead of destroy and recreate the decoder. If ulMaxWidth or ulMaxHeight is set to 0, ulMaxWidth and ulMaxHeight are set to ulWidth and ulHeight respectively.
- ▶ **ChromaFormat**: must be from enum `cudaVideoChromaFormat`. It represents chroma format of content like 4:2:0, 4:4:4, etc.
- ▶ **bitDepthMinus8**: bit-depth minus 8 of video stream to be decoded like 0 for 8-bit, 2 for 10-bit, 4 for 12-bit.
- ▶ **ulNumDecodeSurfaces**: Referred to as decode surfaces elsewhere in this document, this is the number of surfaces that the driver will internally allocate for storing the decoded frames. Using a higher number ensures better pipelining but increases GPU memory consumption. For correct operation, minimum value is defined in `CUVIDEOFORMAT : min_num_decode_surfaces` and can be obtained from first sequence callback from NVIDIA parser. The NVDEC engine writes decoded data to one of these surfaces. These surfaces are not accessible by the user of NVDECODER API, but the *mapping* stage, which includes decoder output format conversion, scaling, cropping etc.) use these surfaces as input surfaces.
- ▶ **ulNumOutputSurfaces**: This is the maximum number of output surfaces that the client will simultaneously *map* to decode surfaces for further processing using `cuidMapVideoFrame()`. These surfaces have postprocessed decoded output to be used by client. The driver internally allocates the corresponding number of surfaces (referred as output surfaces in this document). Client will have access to output surfaces. Refer to section *Preparing the decoded frame for further processing* to understand the definition of *map*.

- ▶ **OutputFormat**: Output surface format defined as enum `cudaVideoSurfaceFormat`. This output format must be one of supported format obtained in `decodecaps.nOutputFormatMask` in `cuidGetDecoderCaps()`. If an unsupported output format is passed, API will fail with error `CUDA_ERROR_NOT_SUPPORTED`.
- ▶ **ulTargetWidth, ulTargetHeight**: This is resolution of output surfaces. For use-case which involve no scaling, these should be set `uWidth, uHeight`, respectively.
- ▶ **DeinterlaceMode**: This should be set to `cudaVideoDeinterlaceMode_Weave` or `cudaVideoDeinterlaceMode_Bob` for progressive content and `cudaVideoDeinterlaceMode_Adaptive` for interlaced content. `cudaVideoDeinterlaceMode_Adaptive` yields better quality but increases memory consumption.
- ▶ **ulCreationFlags**: It is defined as enum `cudaVideoCreateFlags`. It is optional to explicitly define this flag. Driver will pick appropriate mode if not defined.
- ▶ **ulIntraDecodeOnly**: Set this flag to 1 to instruct the driver that the content being decoded contains only I/IDR frames. This helps the driver optimize memory consumption. Do not set this flag if content has non-intra frames.
- ▶ **enableHistogram**: Set this flag to 1 to enable histogram data collection.

The `cuidCreateDecoder()` call fills `CUvideodecoder` with the decoder handle which should be retained till the decode session is active. The handle needs to be passed along with other NVDEC API calls.

The user can also specify the following parameters in the `CUVIDDECODERCREATEINFO` to control the final output:

- ▶ Scaling dimension
- ▶ Cropping dimension
- ▶ Dimension if the user wants to change the aspect ratio

The following code demonstrates the setup of decoder in case of scaling, cropping, or aspect ratio conversion.

```
// Scaling. Source size is 1280x960. Scale to 1920x1080.
CUresult rResult;
unsigned int uScaleW, uScaleH;
uScaleW = 1920;
uScaleH = 1080;
...
CUVIDDECODERCREATEINFO stDecodeCreateInfo;
memset(&stDecodeCreateInfo, 0, sizeof(CUVIDDECODERCREATEINFO));
... // Setup the remaining structure members
stDecodeCreateInfo.ulTargetWidth = uScaleW;
stDecodeCreateInfo.ulTargetHeight = uScaleH;
rResult = cuidCreateDecoder(&hDecoder, &stDecodeCreateInfo);
...
```

```
// Cropping. Source size is 1280x960
CUresult rResult;
unsigned int uCropL, uCropR, uCropT, uCropB;
uCropL = 30;
uCropR = 700;
uCropT = 20;
uCropB = 500;
```

(continues on next page)

(continued from previous page)

```

...
CUVIDDECODERCREATEINFO stDecodeCreateInfo;
memset(&stDecodeCreateInfo, 0, sizeof(CUVIDDECODERCREATEINFO));
// Setup the remaining structure members
...
stDecodeCreateInfo.display_area.left   = uCropL;
stDecodeCreateInfo.display_area.right  = uCropR;
stDecodeCreateInfo.display_area.top    = uCropT;
stDecodeCreateInfo.display_area.bottom = uCropB;
rResult = cuvidCreateDecoder(&hDecoder, &stDecodeCreateInfo);
...

// Aspect Ratio Conversion. Source size is 1280x960(4:3). Convert to
// 16:9
CUresult rResult;
unsigned int uDispAR_L, uDispAR_R, uDispAR_T, uDispAR_B;
uDispAR_L = 0;
uDispAR_R = 1280;
uDispAR_T = 70;
uDispAR_B = 790;
...
CUVIDDECODERCREATEINFO stDecodeCreateInfo;
memset(&stDecodeCreateInfo, 0, sizeof(CUVIDDECODERCREATEINFO));
... // setup structure members
stDecodeCreateInfo.target_rect.left   = uDispAR_L;
stDecodeCreateInfo.target_rect.right  = uDispAR_R;
stDecodeCreateInfo.target_rect.top    = uDispAR_T;
stDecodeCreateInfo.target_rect.bottom = uDispAR_B;
rResult = cuvidCreateDecoder(&hDecoder, &stDecodeCreateInfo);
...

```

4.2.3. Decoding the frame/field

After de-muxing and parsing, the client can submit the bitstream which contains a frame or field of data to hardware for decoding. To accomplish this, the following steps need to be followed:

- ▶ Fill up the CUVIDPICPARAMS structure.
 - ▶ The client needs to fill up the structure with parameters derived during the parsing process. CUVIDPICPARAMS contains a structure specific to every supported codec which should also be filled up.
- ▶ Call `cuvidDecodePicture()` and pass the decoder handle and the pointer to CUVIDPICPARAMS. `cuvidDecodePicture()` kicks off the decoding on NVDEC.

4.2.4. Preparing the decoded frame for further processing

The user needs to call `cuvidMapVideoFrame()` to get the CUDA device pointer and pitch of the output surface that holds the decoded and post-processed frame.

Please note that `cuvidDecodePicture()` instructs the NVDEC hardware engine to kick off the decoding of the frame/field. However, successful completion of `cuvidMapVideoFrame()` indicates that

the decoding process is completed and that the decoded YUV frame is converted from the format generated by NVDEC to the YUV format specified in `CUVIDDECODECREATEINFO::OutputFormat`.

`cuidMapVideoFrame()` API takes decode surface index (`nPicIdx`) as input and *maps* it to one of available output surfaces, post-processes the decoded frame and copy to output surface and returns CUDA device pointer and associated pitch of the output surfaces.

The above operation performed by `cuidMapVideoFrame()` is referred to as *mapping* in this document.

After the user is done with the processing on the frame, `cuidUnmapVideoFrame()` must be called to make the output surface available for storing other decoded and post-processed frames.

If the user continuously fails to call the corresponding `cuidUnmapVideoFrame()` after `cuidMapVideoFrame()`, then `cuidMapVideoFrame()` will eventually fail. At most `CUVIDDECODECREATEINFO::ulNumOutputSurfaces` frames can be mapped at a time.

`cuidMapVideoFrame()` is a blocking call as it waits for decoding to complete. If `cuidMapVideoFrame()` is called on same CPU thread as `cuidDecodePicture()`, it will block `cuidDecodePicture()` as well. In this case, the application will not be able to submit decode packets to NVDEC until mapping is complete. It can be avoided by performing the mapping operation on a CPU thread (referred as mapping thread) different from the one calling `cuidDecodePicture()` (referred as decoding thread).

When using NVIDIA parser from NVDEC API, the application can implement a producer-consumer queue between decoding thread (as producer) and mapping thread (as consumer). The queue can contain picture indexes (or other unique identifiers) for frames being decoded. Parser can run on decoding thread. Decoding thread can add the picture index to the queue in display callback and return immediately from callback to continue decoding subsequent frames as they become available. On the other side, mapping thread will monitor the queue. If it sees the queue has non-zero length, it will dequeue the entry and call `cuidMapVideoFrame(...)` with `nPicIdx` as the picture index. Decoding thread must ensure to not reuse the corresponding decode picture buffer for storing the decoded output until its entry is consumed and freed by mapping thread.

The following code demonstrates how to use `cuidMapVideoFrame()` and `cuidUnmapVideoFrame()`.

```
// MapFrame: Call cuidMapVideoFrame and get the devptr and associated
// pitch. Copy this surface (in device memory) to host memory using
// CUDA device to host memcpy.
bool MapFrame()
{
    CUVIDPARSEDISPINFO stDispInfo;
    CUVIDPROC_PARAMS stProcParams;
    CUresult rResult;
    unsigned long long cuDevPtr = 0;
    int nPitch, nPicIdx, frameSize;
    unsigned char* pHostPtr = nullptr;
    memset(&stDispInfo, 0, sizeof(CUVIDPARSEDISPINFO));
    memset(&stProcParams, 0, sizeof(CUVIDPROC_PARAMS));
    /*****
    *   setup stProcParams
    *****/
    // retrieve the frames from the Frame Display Queue. This Queue is
    // is populated in HandlePictureDisplay.
    if (g_pFrameQueue->dequeue(&stDispInfo))
    {
```

(continues on next page)

(continued from previous page)

```

    nPicIdx = stDispInfo.picture_index;
    rResult = cuvidMapVideoFrame(hDecoder, nPicIdx, &cuDevPtr,
                                &nPitch, &stProcParams);
    frameSize = (ChromaFormat == cudaVideoChromaFormat_444) ? nPitch *
→(3*nheight) :
    nPitch * (nheight + (nheight + 1) / 2);
    // use CUDA based Device to Host memcpy
    rResult = cuMemAllocHost((void**) &pHostPtr, frameSize);
    if (pHostPtr)
    {
        rResult = cuMemcpyDtoH(pHostPtr, cuDevPtr, frameSize);
    }
    rResult = cuvidUnmapVideoFrame(hDecoder, cuDevPtr);
}
... // Dump YUV to a file
if (pHostPtr)
{
    cuMemFreeHost(pHostPtr);
}
...
}

```

In multi-instance decoding use-case, NVDEC could be a bottleneck so there wouldn't be significant benefit of calling `cuvidMapVideoFrame()` and `cuvidDecodePicture()` on different CPU threads. `cuvidDecodePicture()` will stall if wait queue on NVDEC inside driver is full. Sample applications in Video Codec SDK are using *mapping* and *decode* calls on same CPU thread, for simplicity.

4.2.5. Getting histogram data buffer

Histogram data is collected by NVDEC during the decoding process resulting in zero performance penalty. NVDEC computes the histogram data for only the luma component of decoded output, not on post-processed frame(i.e. when scaling, cropping, etc. applied). In case of AV1 when film grain is enabled, histogram data is collected on the decoded frame prior to the application of the film grain.

`cuvidMapVideoFrame()` API returns the CUDA device pointer of histogram data buffer along with output surface if `CUVIDDECODECREATEINFO::enableHistogram` flag is set while creating decoder (using API `cuvidCreateDecoder()`). CUDA device pointer of histogram buffer can be obtained from `CUVIDPROCPARAMS::histogram_dptr`.

Histogram buffer is mapped to output buffer in driver so `cuvidUnmapVideoFrame()` does unmap of histogram buffer also along with output surface.

The following code demonstrates how to use `cuvidMapVideoFrame()` and `cuvidUnmapVideoFrame()` for accessing histogram buffer.

```

// MapFrame: Call cuvidMapVideoFrame and get the output frame and associated
// histogram buffer CUDA device pointer

CUVIDPROCPARAMS stProcParams;
CUsresult rResult;
unsigned long long cuOutputFramePtr = 0, cuHistogramPtr = 0;
int nPitch;
int histogram_size = (decodecaps.nCounterBitDepth / 8) *

```

(continues on next page)

(continued from previous page)

```

                                decodecaps.nMaxHistogramBins;
    unsigned char *pHistogramPtr = nullptr;

    memset(&stProcParams, 0, sizeof(CUVIDPROCPARAMS));
    /*****
    *   setup stProcParams
    *****/
    stProcParams.histogram_dptr = &cuHistogramPtr;

    rResult = cuvidMapVideoFrame(hDecoder, nPicIdx, &cuOutputFramePtr,
                                &nPitch, &stProcParams);
    // allocate histogram buffer for cuMemcpy
    rResult = cuMemAllocHost((void**) &pHistogramPtr, histogram_size);
    if (pHistogramPtr)
    {
        rResult = cuMemcpyDtoH(pHistogramPtr, cuHistogramPtr, histogram_size);
    }
    // unmap output frame
    rResult = cuvidUnmapVideoFrame(hDecoder, cuOutputFramePtr);
    ...
}

```

4.2.6. Querying the decoding status

After the decoding is kicked off, `cuvidGetDecodeStatus()` can be called at any time to query the status of decoding of that frame. The underlying driver fills the status of decoding in `CUVIDGETDECODESTATUS::decodeStatus`.

The NVDEC API currently reports the following statuses:

- ▶ Decoding is in progress.
- ▶ Decoding of the frame completed successfully.
- ▶ The bitstream for the frame was corrupted and concealed by NVDEC.
- ▶ The bitstream for the frame was corrupted, however could not be concealed by NVDEC.

The API is expected to help in the scenarios where the client needs to take a further decision based on the decoding status of the frame, for e.g. whether to carry out inferencing on the frame or not.

Please note that the NVDEC can detect a limited number of errors depending on the codec. This API is supported for HEVC, H264 and JPEG on Turing and above generation GPUs.

4.2.7. Reconfiguring the decoder

Using `cuvidReconfigureDecoder()` the user can reconfigure the decoder if there is a change in the resolution and/or post processing parameters of the bitstream without having to destroy the ongoing decoder instance, and create a new one thereby saving time (and latency) in the process.

In the earlier SDKs the user had to destroy the existing decoder instance and create a new decoder instance for handling any change in decoder resolution or post processing parameters (like scaling ratio, cropping dimensions etc.).

The API can be used in scenarios where the bitstream undergoes changes in resolution, for e.g. when the encoder (on server side) changes image resolution frequently to adhere to Quality of Service(QoS) constraints.

The following steps need to be followed for using the `cuvIdReconfigureDecoder()`.

1. The user needs to specify `CUVIDDECODECREATEINFO::ulMaxWidth` and `CUVIDDECODECREATEINFO::ulMaxHeight` while calling `cuvIdCreateDecoder()`. The user should choose the values of `CUVIDDECODECREATEINFO::ulMaxWidth` and `CUVIDDECODECREATEINFO::ulMaxHeight` which to ensure that the resolution of the bitstream is never exceeded during the entire decoding process. Please note that the values of `CUVIDDECODECREATEINFO::ulMaxWidth` and `CUVIDDECODECREATEINFO::ulMaxHeight` cannot be changed within a session and if user wants to change the values, the decoding session should be destroyed and recreated.
2. During the process of decoding, when the user needs to change the bitstream or change post-processing parameters, the user needs to call `cuvIdReconfigureDecoder()`. This call should be ideally made from `CUVIDPARSERPARAMS::pfnSequenceCallback` when the bitstream changes. The parameters the user wants to reconfigure should be filled up in `:CUVIDRECONFIGUREDECODERINFO`. Please note, `CUVIDRECONFIGUREDECODERINFO::ulWidth` and `CUVIDRECONFIGUREDECODERINFO::ulHeight` must be equal to or smaller than `CUVIDDECODECREATEINFO::ulMaxWidth` and `CUVIDDECODECREATEINFO::ulMaxHeight` respectively or else the `cuvIdReconfigureDecoder()` would fail.

The API is supported for all codecs supported by NVDECODAPI.

4.2.8. Destroying the decoder

The user needs to call `cuvIdDestroyDecoder()` to destroy the decoder session and free up all the allocated decoder resources.

4.3. Run-time dynamic linking of NVIDIA libraries

Video Codec SDK sample applications are using two main NVIDIA libraries: `nvcuvid` and `cuda`. Both libraries can be used as either load-time dynamic linking or run-time dynamic linking. Video Codec SDK sample applications are using load-time dynamic linking. User can use run-time dynamic linking of these libraries if needed. Below code snippets can help understand the changes needed in programming style:

4.3.1. Run-time dynamic linking

In case of run-time dynamic linking, library is loaded to memory at run-time. Below is code snippet to dynamically load `nvcuvid` library at run-time on Windows and Linux systems:

```
#if defined(WIN32) || defined(_WIN32) || defined(WIN64) || defined(_WIN64)
#include <Windows.h>

#ifdef UNICODE
    static LPCWSTR __DriverLibName = L"nvcuvid.dll";
#else
    static LPCSTR __DriverLibName = "nvcuvid.dll";
```

(continues on next page)

(continued from previous page)

```

#endif

typedef HMODULE DLLDRIVER;

static CUresult LOAD_LIBRARY(DLLDRIVER *pInstance)
{
    *pInstance = LoadLibrary(__DriverLibName);

    if (*pInstance == NULL)
    {
        printf("LoadLibrary \"%s\" failed!\n", __DriverLibName);
        return CUDA_ERROR_UNKNOWN;
    }

    return CUDA_SUCCESS;
}

#elif defined(__unix__) || defined(__APPLE__) || defined(__MACOSX)
#include <dlfcn.h>

static char __DriverLibName[] = "libnvcuvid.so";

typedef void *DLLDRIVER;

static CUresult LOAD_LIBRARY(DLLDRIVER *pInstance)
{
    *pInstance = dlopen(__DriverLibName, RTLD_NOW);

    if (*pInstance == NULL)
    {
        printf("dlopen \"%s\" failed!\n", __DriverLibName);
        return CUDA_ERROR_UNKNOWN;
    }

    return CUDA_SUCCESS;
}
#endif

```

4.3.2. Getting function pointers

Function pointers can be fetched using `GetProcAddress()` on Windows and `dlsym()` on Linux:

```

typedef CUresult CUDAAPI tcuvidCreateVideoParser(CUvideoparser *pObj,
    ↪CUVIDPARSERPARAMS *pParams);
typedef CUresult CUDAAPI tcuvidParseVideoData(CUvideoparser obj,
    ↪CUVIDSOURCEDATAPACKET *pPacket);
typedef CUresult CUDAAPI tcuvidDestroyVideoParser(CUvideoparser obj);

typedef CUresult CUDAAPI tcuvidGetDecoderCaps(CUVIDDECODECAPS *pdc);
typedef CUresult CUDAAPI tcuvidCreateDecoder(CUvideodecoder *phDecoder,

```

(continues on next page)

(continued from previous page)

```

↪CUVIDDECODECREATEINFO *pdci);
typedef CUresult CUDAAPI tcuvidDestroyDecoder(CUvideodecoder hDecoder);
typedef CUresult CUDAAPI tcuvidDecodePicture(CUvideodecoder hDecoder,
↪CUVIDPICPARAMS *pPicParams);

tcuvidCreateVideoParser          *cuvidCreateVideoParser;
tcuvidParseVideoData            *cuvidParseVideoData;
tcuvidDestroyVideoParser        *cuvidDestroyVideoParser;

tcuvidGetDecoderCaps            *cuvidGetDecoderCaps;
tcuvidCreateDecoder             *cuvidCreateDecoder;
tcuvidDestroyDecoder            *cuvidDestroyDecoder;
tcuvidDecodePicture             *cuvidDecodePicture;

#if defined(WIN32) || defined(_WIN32) || defined(WIN64) || defined(_WIN64)
#define GET_PROC_EX(name, alias, required) \
    alias = (t##name *)GetProcAddress(DriverLib, #name); \
    if (alias == NULL && required) { \
        printf("Failed to find required function \"%s\" in %s\n", \
            #name, __DriverLibName); \
        return CUDA_ERROR_UNKNOWN; \
    }

#elif defined(__unix__) || defined(__APPLE__) || defined(__MACOSX)
#define GET_PROC_EX(name, alias, required) \
    alias = (t##name *)dlsym(DriverLib, #name); \
    if (alias == NULL && required) { \
        printf("Failed to find required function \"%s\" in %s\n", \
            #name, __DriverLibName); \
        return CUDA_ERROR_UNKNOWN; \
    }
#endif

#define GET_PROC_REQUIRED(name) GET_PROC_EX(name, name, 1)
#define GET_PROC_OPTIONAL(name) GET_PROC_EX(name, name, 0)
#define GET_PROC(name)          GET_PROC_REQUIRED(name)

#define CHECKED_CALL(call) \
    do { \
        CUresult result = (call); \
        if (CUDA_SUCCESS != result) { \
            return result; \
        } \
    } while(0)

CUresult CUDAAPI cuvidInit(unsigned int Flags)
{
    DLLDRIVER DriverLib;

    CHECKED_CALL(LOAD_LIBRARY(&DriverLib));

```

(continues on next page)

(continued from previous page)

```
// fetch all function pointers
GET_PROC(cuvidCreateVideoParser);
GET_PROC(cuvidParseVideoData);
GET_PROC(cuvidDestroyVideoParser);

GET_PROC(cuvidGetDecoderCaps);
GET_PROC(cuvidCreateDecoder);
GET_PROC(cuvidDestroyDecoder);
GET_PROC(cuvidDecodePicture);

// fetch other functions pointers

return CUDA_SUCCESS;
}
```

4.4. Writing an Efficient Decode Application

The NVDEC engine on NVIDIA GPUs is a dedicated hardware block, which decodes the input video bitstream in supported formats. A typical video decode application broadly consists of the following stages:

1. De-Muxing
2. Video bitstream parsing and decoding
3. Preparing the frame for further processing

Of these, de-muxing and parsing are not hardware accelerated and therefore outside the scope of this document. The de-muxing can be performed using third party components such as FFmpeg, which provides support for many multiplexed video formats. The sample applications included in the SDK demonstrate de-muxing using FFmpeg.

Similarly, post-decode or video post-processing (such as scaling, color space conversion, noise reduction, color enhancement etc.) can be effectively performed using user-defined CUDA kernels.

The post-processed frames can then be sent to the display engine for displaying on the screen, if required. Note that this operation is outside the scope of NVDEC API.

An optimized implementation should use independent threads for de-muxing, parsing, bitstream decode and processing etc. as explained below:

1. De-muxing: This thread demultiplexes the media file and makes the raw bit-stream available for parser to consume.
2. Parsing and decoding: This thread does the parsing of the bitstream and kicks off decoding by calling `cuvidDecodePicture()`.
3. Mapping and making the frame available for further processing: This thread checks if there are any decoded frames available. If yes, then it should call `cuvidMapVideoFrame()` to get the CUDA device pointer and pitch of the frame. The frame can then be used for further processing.

The NVDEC driver internally maintains a queue of 4 frames for efficient pipelining of operations. Please note that this pipeline does not imply any decoding delay for decoding. The decoding starts as soon as the first frame is queued, but the application can continue queuing up input frames so long as space is available without stalling. Typically, by the time application has queued 2-3 frames, decoding of the

first frame is complete and the pipeline continues. This pipeline ensures that the hardware decoder is utilized to the maximum extent possible.

For performance intensive and low latency video codec applications, ensure the PCIe link width is set to the maximum available value. PCIe link width currently configured can be obtained by running command 'nvidia-smi -q'. PCIe link width can be configured in the system's BIOS settings.

In the use cases where there is frequent change of decode resolution and/or post processing parameters, it is recommended to use `cuidReconfigureDecoder()` instead of destroying the existing decoder instance and recreating a new one.

The following steps should be followed for optimizing video memory usage:

1. Make `CUVIDDECODERCREATEINFO::ulNumDecodeSurfaces` = `CUVIDEOFORMAT::min_num_decode_surfaces`. This will ensure that the underlying driver allocates minimum number of decode surfaces to correctly decode the sequence. In case there is reduction in decoder performance, clients can slightly increase `CUVIDDECODERCREATEINFO::ulNumDecodeSurfaces`. It is therefore recommended to choose the optimal value of `CUVIDDECODERCREATEINFO::ulNumDecodeSurfaces` to ensure right balance between decoder throughput and memory consumption.
2. `CUVIDDECODERCREATEINFO::ulNumOutputSurfaces` should be decided optimally after due experimentation for balancing decoder throughput and memory consumption.
3. `CUVIDDECODERCREATEINFO::DeinterlaceMode` should be set "`cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Weave`" or "`cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Bob`". For interlaced contents, choosing `cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Adaptive` results to higher quality but increases memory consumption. Using `cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Weave` or `cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Bob` results to minimum memory consumption though it may result in lesser video quality. In case "`CUVIDDECODERCREATEINFO::DeinterlaceMode`" is not specified by the client, the underlying display driver sets it to "`cudaVideoDeinterlaceMode::cudaVideoDeinterlaceMode_Adaptive`" which results to higher memory consumption. Hence it is strongly recommended to choose the right value of `CUVIDDECODERCREATEINFO::DeinterlaceMode` depending on the requirement.
4. While decoding multiple streams it is recommended to allocate minimum number of CUDA contexts and share it across sessions. This saves the memory overhead associated with the CUDA context creation.
5. `CUVIDDECODERCREATEINFO::ulIntraDecodeOnly` should be set to 1 if it is known beforehand that the sequence contains Intra frames only. This feature is supported only for HEVC, H.264 and VP9. However, decoding might fail if the flag is enabled in case of supported codecs for regular bit streams having P and/or B frames.

The sample applications included with the Video Codec SDK are written to demonstrate the functionality of various APIs, but they may not be fully optimized. Hence programmers are strongly encouraged to ensure that their application is well-designed, with various stages in the decode-postprocess-display pipeline structured in an efficient manner to achieve desired performance and memory consumption.

4.5. Allocating decode surfaces dynamically

The bitstream sequence (SPS) often has more DPB size than necessary. In most instances, not all of these surfaces are required. Therefore, the following approach can be used to reduce the number of decode surfaces used during a decode session. This will help lower the memory footprint of each

session in most instances, allowing for an increase in the number of simultaneous decode sessions. However, the decode throughput of individual sessions may be slightly reduced, as the surfaces will be reused more frequently.

The following steps should be followed for further optimizing video memory usage:

1. Set `CUVIDPARSERPARAMS::bMemoryOptimize = 1` before invoking `cuidCreateVideoParser()`. This will ensure that the parser sends lowest possible `CUVIDPICPARAMS::CurrPicIdx`.
2. `pfnSequenceCallback()` has `min_num_decode_surfaces` information. Return `min_num_decode_surfaces` value from `pfnSequenceCallback()`. This will overwrite DPB size to `min_num_decode_surfaces`.
3. Assign a value to `CUVIDDECODECREATEINFO::ulNumDecodeSurfaces` that is less than `min_num_decode_surfaces`, with the minimum value being 1. Next, create a decoder using `cuidCreateDecoder()`. This will create a decoder with a reduced number of decode surfaces.
4. In `pfnDecodeCallback()`, check the value of `CUVIDPICPARAMS::CurrPicIdx` sent by the parser.
 - a. If `CUVIDPICPARAMS::CurrPicIdx` is less than `ulNumDecodeSurfaces`, continue with the decode using `cuidDecodePicture()` API.
 - b. If `CUVIDPICPARAMS::CurrPicIdx` is greater than or equal to `ulNumDecodeSurfaces`, then invoke `cuidReconfigureDecoder()` with increased `CUVIDRECONFIGUREDECODERINFO::ulNumDecodeSurfaces` to increase the number of decode surfaces. Then, proceed with decoding the frame using `cuidDecodePicture()`. Note: If `cuidReconfigureDecoder()` fails with `CUDA_ERROR_OUT_OF_MEMORY` error, application can retry for some time, if `cuidReconfigureDecoder()` continues to fail, the application needs to handle the failure.

4.6. Getting decode statistics from the decoder

The NVDEC API provides the ability to retrieve detailed decode statistics for each decoded frame. These statistics include per 16x16 block information such as quantization parameters, motion vectors, and coding unit types. This feature is useful in video analytics use cases. The API supports this feature for HEVC (H.265) and H.264 codecs.

4.6.1. Enabling decode statistics

To enable decode statistics, follow these steps:

1. **Check decoder capabilities:** Before enabling decode statistics, verify that the decoder supports this feature by calling `cuidGetDecoderCaps()` and checking the `bIsDecodeStatsSupported` field in the `CUVIDDECODECAPS` structure.
2. **Enable the feature during decoder creation:** When creating the decoder with `cuidCreateDecoder()`, set the `enableDecodeFeatures` field in the `CUVIDDECODECREATEINFO` structure to include the `cuidDecodeFeature_DecStats` flag:

```
CUVIDDECODECREATEINFO videoDecodeCreateInfo = {};
// ... other initialization ...
if (decoderCaps.bIsDecodeStatsSupported) {
    videoDecodeCreateInfo.enableDecodeFeatures = (1 << cuidDecodeFeature_
```

(continues on next page)

(continued from previous page)

```

    →DecStats);
}

```

3. **Driver allocates GPU memory for statistics:** The driver allocates GPU memory to store the decode statistics. The size of each statistics buffer is calculated as shown below. The driver allocates N such buffers, where $N = \text{CUVIDRECONFIGUREDECODERINFO}::\text{uNumDecodeSurfaces}$.

```

unsigned int decodeStatsSize = sizeof(CUVIDDECODESTATSHEADER) +
                               ((width + 15) >> 4) * ((height + 15) >> 4)
    →* sizeof(CUVIDDECODESTATS);

```

Where: - width and height are the frame dimensions - Each 16x16 macroblock requires one CUVIDDECODESTATS structure - The total includes a header structure CUVIDDECODESTATSHEADER

4.6.2. Retrieving decode statistics

The decode statistics are retrieved during the frame mapping process using `cuidMapVideoFrame()`:

1. **Set up the extended processing parameters:** Create a `CUVIDPROCPARAMSEXT` structure and set the `decode_stats_dptra` field to point to a variable that will receive the GPU memory address containing the statistics:

```

CUVIDPROCPARAMSEXT videoProcessingParametersExt = {};
CUdeviceptr decodeStatsDptr = 0;
videoProcessingParametersExt.decode_stats_dptra = &decodeStatsDptr;

```

2. **Attach to the main processing parameters:** Link the extended parameters to the main `CUVIDPROCPARAMS` structure:

```

CUVIDPROCPARAMS videoProcessingParameters = {};
// ... other initialization ...
videoProcessingParameters.pCuidProcExt = &videoProcessingParametersExt;

```

3. **Map the video frame:** Call `cuidMapVideoFrame()` with the processing parameters. After successful mapping, the `decodeStatsDptr` variable will contain the GPU memory address where the statistics are stored:

```

CUdeviceptr dpSrcFrame = 0;
unsigned int nSrcPitch = 0;
cuidMapVideoFrame(hDecoder, pictureIndex, &dpSrcFrame, &nSrcPitch, &
    →videoProcessingParameters);

```

4. **Copy statistics to host memory or process directly:** Use `cuMemcpyDtoH()` to copy the statistics from GPU memory to host memory for CPU-based analysis, or process the statistics directly in GPU memory using CUDA kernels for real-time applications. The GPU memory address is provided in the `decode_stats_dptra` variable after successful frame mapping.

```

if (decodeStatsDptr != 0) {
    uint8_t* pDecodeStats = new uint8_t[decodeStatsSize];
    cuMemcpyDtoH(pDecodeStats, decodeStatsDptr, decodeStatsSize);
    // Process the statistics...
    delete[] pDecodeStats;
}

```

4.6.3. Decode statistics structure

The decode statistics buffer contains two main components:

1. Header Structure (CUVIDDECODESTATSHEADER)

The header structure provides metadata about the decode statistics:

```
typedef struct _CUVIDDECODESTATSHEADER
{
    unsigned char bIsStatsPresent : 1;    // Flag to indicate whether decode
    ↪ stats dump is present or not.
    unsigned char bIsMV1Valid : 1;        // Flag to indicate if MV1 is valid
    ↪ (set to 1 if valid)
    unsigned char reserved1 : 6;           // Reserved : shall be set to 0
    unsigned char reserved2[3];           // Reserved : shall be set to 0
    unsigned int  mbCount;                 // Total number of 16x16 blocks in
    ↪ current frame
    unsigned int  size;                    // Size of decode stats data of all
    ↪ blocks (for sanity check)
    unsigned int  reserved3[5];           // Reserved : shall be set to 0
} CUVIDDECODESTATSHEADER;
```

Field Descriptions:

- ▶ **bIsStatsPresent:** Boolean flag indicating whether decode statistics are present for this frame
- ▶ **bIsMV1Valid:** Boolean flag indicating if MV1 is valid (set to 1 if valid)
- ▶ **mbCount:** Total number of 16x16 macroblocks in the current frame (calculated as $((width + 15) >> 4) * ((height + 15) >> 4)$)
- ▶ **size:** Size of the decode statistics data in bytes (for validation purposes). The size of the decode statistics data of all blocks is $mbCount * sizeof(CUVIDDECODESTATS)$.

2. Per-Macroblock Statistics (Array of CUVIDDECODESTATS)

Following the header, the buffer contains an array of *CUVIDDECODESTATS* structures, one for each 16x16 macroblock in raster scan order:

```
typedef struct _CUVIDDECODESTATS
{
    unsigned char qp_luma;                 // Quantization Parameter for luma
    ↪ component
    unsigned char qp_invalid : 1;          // 0: QP Valid, 1: QP is invalid
    unsigned char cu_type : 3;             // Coding Unit type
    unsigned char reserved1 : 4;           // Reserved for alignment/future use
    unsigned char reserved2[2];            // Reserved for alignment/future use
    short mv0_x;                           // First motion vector X component
    short mv0_y;                           // First motion vector Y component
    short mv1_x;                           // Second motion vector X component
    short mv1_y;                           // Second motion vector Y component
} CUVIDDECODESTATS;
```

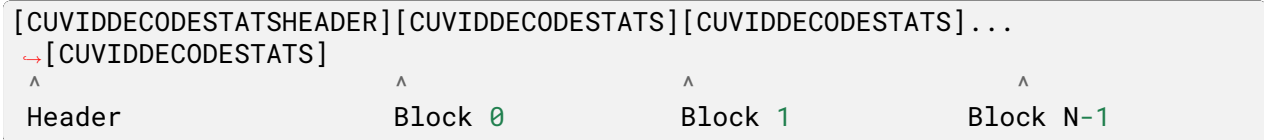
Field Descriptions:

- ▶ **qp_luma:** Quantization parameter for luma component * For H.264/HEVC: $QP_Y + Bitdepth - QP - ScaleFactor * Bitdepth - QP - ScaleFactor = 0$ for 8-bit content, 12 for 10-bit content, 24 for 12-bit content

- ▶ **qp_invalid**: Flag indicating if QP is invalid (0: Valid, 1: Invalid)
- ▶ **cu_type**: Coding unit type (3-bit field): * 0: Intra-coded block * 1: Inter-coded block * 2: Skip block * 3: PCM block * 7: Invalid block
- ▶ **mv0_x, mv0_y**: First motion vector components * H.264: Forward motion vector if valid, otherwise backward motion vector * HEVC: Forward motion vector
- ▶ **mv1_x, mv1_y**: Second motion vector components (for B-frames) * H.264: Unused (set to 0) * HEVC: Backward motion vector

Memory Layout:

The complete buffer layout is:



Where $N = mbCount$ from the header.

4.6.4. Sample application flow

The AppDec sample application demonstrates the complete flow for obtaining decode statistics:

1. **Initialization**: The application enables decode statistics during decoder creation by passing the `bEnableDecodeStats` parameter to the `NvDecoder` constructor.
2. **Statistics collection**: During frame processing, the application calls `NvDecoder::GetDecodeStats()` to retrieve statistics for each decoded frame.
3. **File output**: The statistics are written to a binary file for later analysis.

Command to run AppDec with statistics dumping:

```
./AppDec -i input.h264 -o output.yuv -dumpstats stats.bin
```

The `-dumpstats` parameter specifies the output file where decode statistics will be saved in binary format.

4. **Memory management**: The application properly manages the host memory allocated for statistics storage.

The key functions in the sample application include:

- ▶ `NvDecoder::GetDecodeStats()`: Retrieves decode statistics for the next available frame
- ▶ `NvDecoder::GetLockedDecodeStats()`: Retrieves and removes statistics from the internal queue
- ▶ Proper cleanup of allocated memory in the destructor

4.7. Random Frame Access and Seeking

The NVIDIA Video Codec SDK provides comprehensive seeking and random frame access capabilities through the `SeekUtils` class and the high-level `NvVideoDecoder` interface. These features enable efficient frame-by-frame access, batch decoding, and GOP-aware seeking for video analytics and non-linear video processing applications.

The seeking feature supports:

- ▶ **Random frame access:** Decode specific frames by index with GOP-aware optimization
- ▶ **Batch operations:** Efficiently retrieve multiple sequential or arbitrary frames
- ▶ **Backward seeking:** Automatic decoder reset and state management for backward seeks
- ▶ **Open GOP support:** Handle streams with open GOP structure (non-IDR I-frames)
- ▶ **PTS-based optimization:** Skip frames before target frame during seeking
- ▶ **Sequential mode:** Support for non-seekable streams (e.g., network or elementary streams)

4.7.1. Understanding the Seek Architecture

The seeking functionality is built on three main components:

1. SeekUtils Class (Low-Level API)

SeekUtils is the core seeking layer that provides:

- ▶ GOP-aware seeking with keyframe detection
- ▶ Frame caching and pending frame management
- ▶ PTS-based frame filtering
- ▶ Support for both seekable and non-seekable streams

2. NvVideoDecoder Class (High-Level API)

NvVideoDecoder provides a simplified interface built on top of SeekUtils:

- ▶ Operator-based frame access: `decoder[index]` and `decoder[{indices}]`
- ▶ Automatic decoder reconfiguration for multiple video sources
- ▶ Decoder caching for efficient playlist (multiple videos) processing
- ▶ Simplified resource management

3. AppDecVideoDecoder Sample Application

The AppDecVideoDecoder sample demonstrates practical usage:

- ▶ Command-line interface for frame selection
- ▶ Support for frame indices, time-based access, and range syntax
- ▶ Integration with decode statistics
- ▶ Playlist processing with decoder reuse

4.7.2. Using NvVideoDecoder for Frame Access

The NvVideoDecoder class provides the simplest way to access frames by index. Follow these steps:

1. **Create the decoder instance:** Initialize with the video file path and CUDA context:

```
#include "NvVideoDecoder.h"

// Create CUDA context
CUcontext cuContext = NULL;
```

(continues on next page)

(continued from previous page)

```
ck(cuCtxCreate(&cuContext, 0, 0));

// Create decoder with optional scanning for keyframe metadata
NvVideoDecoder decoder("input.mp4",
                        cuContext,
                        NULL,           // CUDA stream (NULL = default)
                        true,          // useDeviceMemory
                        0,              // maxWidth (0 = auto)
                        0,              // maxHeight (0 = auto)
                        true,           // needScannedStreamMetadata
                        0,              // decoderCacheSize
                        false,          // enableDecodeStats
                        false);         // enableOpenGOP
```

2. **Access frames using operator[]:** The operator syntax provides intuitive frame access:

```
// Single frame access
NvDecFrame frame = decoder[100]; // Get frame at index 100

// Multiple frames with vector syntax
std::vector<uint32_t> indices = {10, 20, 30, 40};
std::vector<NvDecFrame> frames = decoder[indices];

// Sequential batch access
decoder.SeekToIndex(50); // Set starting position
std::vector<NvDecFrame> batch = decoder.GetBatchFrames(10); // Get 10
→frames
```

3. **Access frame data:** The NvDecFrame structure contains:

```
struct NvDecFrame {
    CUdeviceptr data;           // GPU frame buffer pointer
    int64_t timestamp;         // Frame timestamp (PTS)
    uint8_t* decodeStats;      // Decode statistics buffer (if enabled)
    uint32_t decodeStatsSize;  // Decode statistics size
    bool isSkipped;            // True if frame was skipped by seekPTS
    →optimization
};
```

4. **Copy frame to host memory** (if needed):

```
// Get frame metadata
StreamMetadata metadata = decoder.GetStreamMetadata();
uint32_t frameSize = metadata.width * metadata.height * 3 / 2; // NV12
→format

// Allocate host memory
uint8_t* hostFrame = new uint8_t[frameSize];

// Copy from GPU to host
ck(cuMemcpyDtoH(hostFrame, frame.data, frameSize));

// Process frame...
```

(continues on next page)

(continued from previous page)

```
// Clean up
delete[] hostFrame;
```

5. **Get stream metadata:** Retrieve video properties:

```
// Basic metadata (fast, no scanning)
StreamMetadata metadata = decoder.GetStreamMetadata();
std::cout << "Width: " << metadata.width << std::endl;
```

6. **Time-based frame access:** Convert timestamps to frame indices:

```
// Get frame at specific time (in seconds)
float timeInSeconds = 5.5f;
uint32_t frameIndex = decoder.GetIndexFromTimeInSeconds(timeInSeconds);
NvDecFrame frame = decoder[frameIndex];
```

Note: Time-based seeking assumes constant frame rate (CFR). For variable frame rate (VFR) videos, the frame index calculation may be inaccurate. In such cases, use frame index-based seeking with `-frameList` or `-f` for precise frame selection.

4.7.3. Enabling Open GOP Support

Open GOP (Group of Pictures) streams contain non-IDR I-frames (CRA frames in HEVC) that may reference frames from previous GOPs. To correctly decode these streams when seeking:

1. **Enable during decoder creation:**

```
NvVideoDecoder decoder("input.mp4",
                        cuContext,
                        NULL,
                        true,
                        0,
                        0,
                        true,    // needScannedStreamMetadata - required for
→ open GOP
                        0,
                        false,
                        true);  // enableOpenGOP
```

2. **Or enable after creation:**

```
decoder.EnableOpenGOPSupport(true);
```

3. **How it works:**

When open GOP support is enabled:

- ▶ The decoder scans for both keyframes (I-frames) and IDR frames
- ▶ For seeking, it uses the nearest IDR frame instead of the nearest keyframe
- ▶ This ensures all reference frames are available for decoding non-IDR I-frames
- ▶ The `IsOpenGOP()` check compares IDR count vs keyframe count to detect open GOP structure

4.7.4. PTS-Based Frame Skipping Optimization

The PTS (Presentation Timestamp) frame skipping feature optimizes seeking performance by skipping frames that come before the target frame. This is enabled by default but can be controlled:

```
// Disable PTS frame skipping (for debugging or compatibility)
decoder.SetPTSFrameSkip(false);

// Check if enabled
bool isEnabled = decoder.IsPTSFrameSkipEnabled();
```

How it works:

1. When seeking to a target frame, SeekUtils calculates the PTS threshold based on the frame before the target
2. This threshold is passed to NvDecoder::setSeekPTS()
3. The decoder marks frames with PTS < threshold as “skipped” (isSkipped = true)
4. SeekUtils automatically unlocks skipped frames without processing them
5. This reduces memory copies and processing overhead for frames that won't be used

Note: Disable this optimization if you encounter issues with streams that have non-standard PTS ordering.

4.7.5. Handling Non-Seekable Streams

For network streams, pipes, or elementary streams (raw H.264/HEVC files without containers) that don't support efficient random access:

1. **Sequential mode is automatic:** SeekUtils detects non-seekable streams using FFmpegDemuxer::IsSeekable()
2. **Limited seeking capabilities:**

```
// Check if stream is seekable
bool isSeekable = decoder.IsSeekable();

if (!isSeekable) {
    // Forward-only seeking is efficient
    // Backward seeking is supported but requires decoder reset
    // and re-decoding from the beginning (non-optimal)
    std::cout << "Stream has limited seeking support" << std::endl;
}
```

3. **Sequential frame access:**

```
// For non-seekable streams, sequential access is most efficient
for (uint32_t i = 0; i < numFrames; i++) {
    NvDecFrame frame = decoder[i];
    // Process frame...
}
```

4. **Limitations:**

- Forward seeking: Efficient (decoder continues from current position)

- ▶ Backward seeking: Supported but non-optimal (requires decoder reset and re-decode from start)
- ▶ GOP-aware optimization is disabled (no keyframe index available)
- ▶ Container formats (MP4, MKV, etc.) provide much better seeking performance

4.7.6. Decoder Reconfiguration for Multiple Videos

NvVideoDecoder supports efficient switching between multiple video sources with automatic decoder caching:

1. Enable decoder caching:

```
// Create decoder with cache size (e.g., 3 decoders)
NvVideoDecoder decoder("video1.mp4",
                        cuContext,
                        NULL,
                        true,
                        0,
                        0,
                        true,
                        3);    // decoderCacheSize = 3
```

2. Reconfigure for new video:

```
// Switch to new video
ReconfigureStatus status = decoder.ReconfigureDecoder("video2.mp4",
→ false);

if (status == ReconfigureStatus::CacheHit) {
    std::cout << "Reused cached decoder" << std::endl;
} else if (status == ReconfigureStatus::CacheMiss) {
    std::cout << "Created new decoder" << std::endl;
} else if (status == ReconfigureStatus::ResetState) {
    std::cout << "Same source, state reset only" << std::endl;
}

// Access frames from new video
NvDecFrame frame = decoder[50];
```

3. Caching mechanism:

- ▶ Decoders are cached by (codec, bitDepth, chromaFormat)
- ▶ LRU eviction when cache is full
- ▶ Significant performance improvement for playlist processing
- ▶ Reduces decoder creation overhead

4. Reconfigure same source (reset state):

```
// Reset to beginning of same video (preserves keyframe cache)
decoder.ReconfigureDecoder("video1.mp4", true); // isSameSource = true
```

4.7.7. Using AppDecVideoDecoder Sample Application

The AppDecVideoDecoder sample demonstrates all seeking features with a comprehensive command-line interface.

Important Note: The `-o` (output file) flag is optional. When not specified, frames are decoded and kept on the GPU without writing to disk, enabling faster decode-only performance measurements.

Command-Line Options:

Basic Usage:

```
# Decode specific frames by index
./AppDecVideoDecoder -i input.mp4 -framelist 0,10,20,30,40

# Decode frames using range syntax (start:end:step) and save to output file
./AppDecVideoDecoder -i input.mp4 -o output.yuv -framelist 0:100:10

# Time-based access
./AppDecVideoDecoder -i input.mp4 -o output.yuv -t 0:10:2

# Playlist mode
./AppDecVideoDecoder -playlist playlist.txt
```

Use `./AppDecVideoDecoder --help` for the complete list of command-line options.

Playlist File Format:

Create a text file with key:value pairs (one per line). Empty lines separate videos:

```
input:video1.mp4
output:video1.yuv
stats:video1.stats
indices:0,10,20,30,40

input:video2.mp4
output:video2.yuv
stats:video2.stats
indices:20:80:10

input:video3.mp4
```

Notes:

- ▶ `input:` is required (specifies input video file)
- ▶ `output:`, `stats:`, and `indices:` are optional
- ▶ Empty lines separate video entries
- ▶ Frame indices support same syntax as `-framelist` (individual indices, ranges, or mixed)
- ▶ Comments start with `#`
- ▶ The `-dumpstats` flag is ignored in playlist mode; use `stats:` per video instead

Frame Index Syntax:

The application supports flexible frame specification:

- ▶ **Individual indices:** 2, 3, 6, 8, 99

- ▶ **Range with step:** 0:100:2 (every 2nd frame from 0 to 100, inclusive)
- ▶ **Range with default step:** 0:100 (every frame from 0 to 100, step=1)
- ▶ **Mixed syntax:** 0:10:2, 20, 30:35, 50
- ▶ **Time-based:** -t 1.5, 3.0, 5.5 (frames at 1.5s, 3.0s, 5.5s)
- ▶ **Time range:** -t 0:10:0.5 (every 0.5 seconds)

Chapter 5. NVDEC Direct Output to Block-Linear CUDA Arrays

NVDEC can decode video frames directly into externally allocated CUDA arrays in an opaque block-linear memory layout. In the traditional decode path, NVDEC outputs to internal pitch-linear surfaces that must be accessed via `cuvvidMapVideoFrame()`/`cuvvidUnmapVideoFrame()` — any downstream consumer (encoder, inference, post-processing) then needs a copy to move the data into its own buffer. With opaque output, NVDEC writes decoded pixels directly into application-provided CUDA arrays, bypassing the internal surface pool and the map/copy step entirely.

5.1. Key Advantages

- ▶ Eliminates the pitch-linear to block-linear copy that would otherwise be done by a CUDA kernel on SMs. This frees SM capacity for other parallel workloads (inference, rendering, image processing).
- ▶ Reduces GPU memory bandwidth consumption — no intermediate surface and no redundant data movement.
- ▶ CUDA arrays in block-linear format are the native input format for NVENC and CUDA texture operations, so downstream consumers can use the decoded output directly.

5.2. Supported Opaque Surface Formats

Table 1: Supported Opaque Surface Formats

Format	Chroma	Bit Depth
<code>cudaVideoSurfaceFormat_NV12_Opaque</code>	4:2:0	8-bit
<code>cudaVideoSurfaceFormat_P016_Opaque</code>	4:2:0	10/12-bit
<code>cudaVideoSurfaceFormat_NV16_Opaque</code>	4:2:2	8-bit
<code>cudaVideoSurfaceFormat_P216_Opaque</code>	4:2:2	10/12-bit
<code>cudaVideoSurfaceFormat_YUV444_Opaque</code>	4:4:4	8-bit
<code>cudaVideoSurfaceFormat_YUV444_16Bit_Opaque</code>	4:4:4	10/12-bit

5.3. Step-by-Step Usage

1. Allocate a pool of CUarray handles to serve as the decoder's output surfaces, sized for the codec's DPB requirements plus headroom.
2. Create the decoder with `OutputFormat` set to an `_Opaque` variant and `ulNumOutputSurfaces = 0`.
3. Register the CUDA arrays with the decoder by calling `cuidRegisterDecodeSurfaces()` after `cuidCreateDecoder()`.
4. Decode using `cuidDecodePictureAsync()` instead of `cuidDecodePicture()`, passing a `CUstream` for stream-ordered synchronization.
5. Consume the decoded CUDA array directly — no `cuidMapVideoFrame()/cuidUnmapVideoFrame()` needed; the frame is already in the registered CUarray.

5.4. Reference: Traditional vs Opaque Data Flow

Traditional (pitch-linear):

```
cuidCreateDecoder(OutputFormat = NV12, ulNumOutputSurfaces = 2)
cuidDecodePicture(N)
cuidMapVideoFrame(N)      CUdeviceptr + pitch
    copy to consumer buffer (SM kernel)
cuidUnmapVideoFrame(N)
```

Opaque (block-linear):

```
cuidCreateDecoder(OutputFormat = NV12_Opaque, ulNumOutputSurfaces = 0)
cuidRegisterDecodeSurfaces(CUarray pool)
cuidDecodePictureAsync(N, cudaStream)    frame is directly in CUarray, ready
→ for consumption.
```

Notices

Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation ("NVIDIA") makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgment, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer ("Terms of Sale"). NVIDIA hereby expressly

objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer's own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer's sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer's product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

Trademarks

NVIDIA, the NVIDIA logo, and cuBLAS, CUDA, CUDA Toolkit, cuDNN, DALI, DIGITS, DGX, DGX-1, DGX-2, DGX Station, DLProf, GPU, Jetson, Kepler, Maxwell, NCCL, Nsight Compute, Nsight Systems, NVcaffe, NVIDIA Deep Learning SDK, NVIDIA Developer Program, NVIDIA GPU Cloud, NVLink, NVSHMEM, PerfWorks, Pascal, SDK Manager, Tegra, TensorRT, TensorRT Inference Server, Tesla, TF-TRT, Triton Inference Server, Turing, and Volta are trademarks and/or registered trademarks of NVIDIA Corporation in the United States and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

©2022-2026, NVIDIA Corporation & affiliates. All rights reserved